LECTURE 24

Deductive Synthesis

Deductive synthesis is the oldest form of program synthesis, and for many years program synthesis was assumed to mean deductive synthesis. The roots of this style of synthesis go back to the work of Burstall and Darrington [1]. In their original paper, they present the idea of starting with a clean version of a recursive program and applying semantics preserving transformations in order to get a new program that is more efficient even if it is unreadable.

At that level of description, it sounds an awful lot like a compiler, and at some level, the line between deductive synthesis and compilers is even blurrier than the already blurry line between synthesis and compilation in general. What makes these systems different from compilers is that the set of transformations required to go from a specification to an implementation, especially the implementation that the user actually wants, is not always straightforward, so unlike compilers, which usually apply transformations in some simple predefined order, these systems usually require a combination of user guidance and sophisticated search techniques in order to arrive at a good implementation (or in some cases to arrive at any implementation at all).

24.1 A Transformation System for Developing Recursive Programs

The original paper by Burstall and Darrington [1] which launched the field of deductive synthesis argued for a system that would allow programmers to write the clean simple version of their implementation and then, by applying a series of transformation rules, convert that implementation into a more efficient one. One of the key observations in the paper was that a few simple rules could, in some cases, even change the asymptotic complexity of a solution.

The starting point in the system is a recursive definition of a simple function. One of the examples given by the paper is the Fibonacci function, defined in terms of the recurrence:

$$\begin{aligned} f(0) &= 1 \\ f(1) &= 1 \\ f(x+2) &= f(x+1) + f(x) \end{aligned}$$

The paper then provides a series of rules that can be used to manipulate such a definition in order to get a more efficient implementation. In the case of this example, the rules can be used to transform the function from exponential to a linear implementation.

The application of the transformation rules is not straightforward, and transforming the program often involves clever steps that may not be obvious at first. For example, in optimizing the fibonacci function, the first clever step, what they call the eureka step, is to introduce a new function

$$g(x) = \langle f(x+1), f(x) \rangle$$

The transformation then proceeds to apply the transformation rules to the definitions of g or f. The key rules are listed below.

- **Instantiation:** This rule just says that whenever you have a function definition of the form $f(x) = expr$, you can instantiate x to some other expression, for example to $f(a+b) = expr[x \rightarrow a+b]$. In the case of this example, this rule is used on $g(x)$ to replace x with $x+1$ to get

$$g(x+1) = \langle f(x+1+1), f(x+1) \rangle = \langle f(x+2), f(x+1) \rangle$$

- **Unfold:** This rule just corresponds to inlining; it replaces a use of a function with its definition. For example, in the instantiation of g above, unfolding the definition of f in $f(x+2)$ yields the definition below.

$$g(x+1) = \langle f(x+1) + f(x), f(x+1) \rangle$$

- **Abstraction:** This rule says that you can give names to sub-expressions, so if you have an expression E with a subexpression F, you can give a fresh name u to the subexpression and replace E with $E[F \rightarrow u]\ where\ u = F$. This rule allows us to transform the definition of g.

$$g(x+1) = \langle u+v, u \rangle\ where\ \langle u, v \rangle = \langle f(x+1), f(x) \rangle$$

- **Fold:** Is the opposite of unfold. This transformation identifies instances of a function body and replaces them with a call to the function. For example, applying unfold to the definition of $g(x+1)$ above transforms it into

$$g(x+1) = \langle u+v, w \rangle\ where\ \langle u, v \rangle = g(x)$$

The rules above are enough to complete the transformation of $f(x)$ into a linear-time implementation. Applying the abstraction rule, the definition of $f(x+2)$ can be rewriten as

$$f(x+2) = u + v \; where \; \langle u, v \rangle = \langle f(x+1), f(x) \rangle$$

Finally, an application of the fold rule gives us a linear time implementation of Fibonacci.

$$f(x+2) = u + v \; where \; \langle u, v \rangle = g(x)$$

.

24.2 Dreams ⇒ Programs

In 1979, waldinger and Manna published an ambitiously titled paper that presented ideas similar to those of Burstall and Darrington but put them on a more formal footing [2]. The paper was also much more ambitious, aiming not just to transform a simple implementation to a more efficient one, but aiming to bridge the gap between truly declarative logical specifications and their efficient implementation. We illustrate the key ideas of the paper by using one of the running examples presented in the paper.

In this example, the goal is to synthesize a function $lesall(x, l)$ that determines whether x is less than all elements in a list l.

$$\begin{array}{rcl} lesall(x, l) & \Leftarrow & \text{compute } x < all(l) \\ & & \text{where } x \text{ is a number and} \\ & & l \text{ is a list of numbers} \end{array}$$

In general, at each point in the derivation, there is a program of the form

$$f(in) \quad \Leftarrow \quad expr$$

where expressions may include some assumptions $expr$ where $assumptions$. This program will be transformed via either conditional or unconditional transformation rules. When the rules are conditional, they will imply a proof obligation to show that the condition in the rule is implied by the assumptions.

Below are a series of rules and their application to this example.

- **Empty lists:** There is a rule that says that for any predicate P,

$$P(all(l)) \Rightarrow true \text{ if } l \text{ is an empty list}$$

The rule could in principle be applied to our specification by making $P(t) = x < t$, however, that would lead to an obligation to prove that l is an empty list, which cannot be proven from the assumptions in the definition of $lesall$.

- **Conditional formation:** We can add more assumptions by using the conditional formation rule. The rule says that whenever we have a program of the form $S\ where\ Q$, we can use any predicate P to split it into two cases.

$$\begin{array}{rcl} S\ where\ Q & \Rightarrow & if(P)\ S\ where\ (P \wedge Q) \\ & & else\ S\ where\ (\neg P \wedge Q) \end{array}$$

In the context of the running example, this means the definition of $lesall$ can be split into two cases

$$\begin{array}{rcl} lesall(x,l) & \Leftarrow & if(empty(l))\ \text{compute}\ x < all(l)\ where\ (empty(l) \wedge Q) \\ & & else\ \text{compute}\ x < all(l)\ where\ (\neg empty(l) \wedge Q) \end{array}$$

where Q is the original assumption that x is a number and l is a list. Application of this rule now makes it possible for us to apply the rule for empty lists to the first case, leaving us with the definition below.

$$\begin{array}{rcl} lesall(x,l) & \Leftarrow & if(empty(l))\ true\ where\ (empty(l) \wedge Q) \\ & & else\ \text{compute}\ x < all(l)\ where\ (\neg empty(l) \wedge Q) \end{array}$$

- **Non-empty lists:** There is another rule for non-empty lists that says that for all P,

$$\begin{array}{rcl} P(all(l)) & \Rightarrow & P(head(l)) \wedge P(all(tail(l))) \\ & & \text{if } l \text{ is a nonempty list.} \end{array}$$

Note that this rule can be applied to the $else$ case of the program. The requirement of the rule that l be nonempty matches the assumption in this else case, so we are left with:

$$\begin{array}{rcl} lesall(x,l) & \Leftarrow & if(empty(l))\ true\ where\ (empty(l) \wedge Q) \\ & & else\ x < head(l) \wedge x < all(tail(l))\ where\ (\neg empty(l) \wedge Q) \end{array}$$

- **Recursive call formation:** Finally, there is a rule that is analogous to the *fold* rule of Burstall and Darrington that allows us to replace an code fragment that matches a function definition to a call for that function. Given a function definition of the form $f(x) \Rightarrow$ compute $P(x)$ $when$ Q, we can perform the replacement

$$P(t) \quad \Rightarrow \quad f(t) \quad \text{if } Q \text{ is satisfied and } f(t) \text{ terminates}$$

  In the case of the running example, Q is the condition from the original definition of $lesall$, which is clearly satisfied by $tail(l)$, so the program can be further transformed to

$$lesall(x,l) \quad \Leftarrow \quad if(empty(l))\ true\ where\ (empty(l) \wedge Q) \\ else\ x < head(l) \wedge lesall(x, tail(l))\ where\ (\neg empty(l) \wedge Q)$$

  At this point, we have a recursive implementation. Termination is guaranteed by the fact that the list passed to the recursive call is smaller than the input list.

The deductive approach explained in the paper has a number of advantages; the most important is that it is *correct by construction*. Unlike, say, the CEGIS approach, which relies on an independent verification step to ensure the correctness of the final artifact, the deductive approach starts with a correct specification, and by applying semantics preserving transformations ensures that at every step the current program is a correct representation of the specification. There are still some checks that need to be performed along the way, to ensure that the conditions in the transformation rules are valid given the available assumptions, but these tend to be local checks that are easy to discharge automatically.

24.3 Modern deductive systems

The correct by construction aspect of deductive synthesis systems has made it very appealing in constructing software with high correctness guarantees. The Kesterel institute, for example, has a number of success stories around the development of complex verified systems such as high performance schedulers [3].

Traditionally, though a few aspects have hampered the large-scale adoption of deductive synthesis systems. The most commonly cited shortcoming is that deductive systems are challenging to use. It is not always obvious which rules to apply, and some rules like the conditional formation rule above, require the

user to know exactly what predicate to use when applying the rule. Sometimes it takes a very large number of transformation steps to get from the high-level specification to the desired implementation, and the user really needs to internalize all the quirks in the transformation system to use it effectively. Moreover, for every new domain, the systems have to be extended with new transformations specific to that domain.

In recent years, though, there have been some significant success stories from deductive synthesis systems. The best known one is the Spiral system [4], which was originally designed for signal processing kernels, including FFTs and other linear transformations, but has more recently been extended to a variety of other domains. Part of the success of the Spiral family of systems has been due to very well designed systems of transformation rules and strategies for how to apply them that are tailored for very narrow domains. The domain specificity makes it possible to incorporate a lot of domain knowledge into the rules and the application strategies, enabling fully automated transformation from high-level specifications to highly efficient implementations. For many domains, they have demonstrated they can generate implementations that are much better even than those hand crafted by performance experts.

A very recent and promising system is the Fiat system, developed by Delaware, Pit–Claudel, Gross and Chlipala [5]. Fiat is built on top of the Coq proof assistant, and leverages the automation that Coq's tactic language makes possible in order to provide a high-degree of automation while raising the level of abstraction at which users need to provide input.

References

[1] R. M. Burstall and John Darlington. A transformation system for developing recursive programs. *J. ACM*, 24(1):44–67, January 1977. ISSN 0004-5411. doi: 10.1145/321992.321996. URL http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.19.4684&rep=rep1&type=pdf.

[2] Z. Manna and R. Waldinger. Synthesis: Dreams ? programs. *IEEE Trans. Softw. Eng.*, 5(4):294–328, July 1979. ISSN 0098-5589. doi: 10.1109/TSE.1979.234198. URL https://ieeexplore.ieee.org/abstract/document/1702636/.

[3] Lee Blaine, Limei Gilham, Junbo Liu, Douglas R. Smith, and Stephen J. Westfold. Planware - domain-specific synthesis of high-performance schedulers. In *The Thirteenth IEEE Conference on Automated Software Engineering, ASE 1998, Honolulu, Hawaii, USA, October 13-16, 1998*, page 270, 1998. doi: 10.1109/ASE.1998.732672. URL https://doi.org/10.1109/ASE.1998.732672.

[4] Markus Püschel, José M. F. Moura, Jeremy R. Johnson, David A. Padua, Manuela M. Veloso, Bryan Singer, Jianxin Xiong, Franz Franchetti, Aca Gacic, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nicholas Rizzolo. SPIRAL: code generation for DSP transforms. *Proceedings of the IEEE*, 93(2):232–275, 2005. doi: 10.1109/JPROC.2004.840306. URL https://doi.org/10.1109/JPROC.2004.840306.

[5] Benjamin Delaware, Clément Pit-Claudel, Jason Gross, and Adam Chlipala. Fiat: Deductive synthesis of abstract data types in a proof assistant. In *Proceedings of the 42Nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '15, pages 689–700, New York, NY, USA, 2015. ACM. ISBN 978-1-4503-3300-9. doi: 10.1145/2676726.2677006. URL http://doi.acm.org/10.1145/2676726.2677006.